LAB 7: Fuzzing for Input Validation Bugs

Software Engineering Lab Report & Reflection Analysis

Student Name:	Sujay Hegde	**SRN / Roll No:**	PES1UG24CS478
Course:	Software Engineering (SE)	**Semester / Sec:**	6th Sem / CSE
Repository:	sujayhegde23/LAB6-LAB7-SE-PES1UG24CS478	**Lab Duration:**	60 minutes

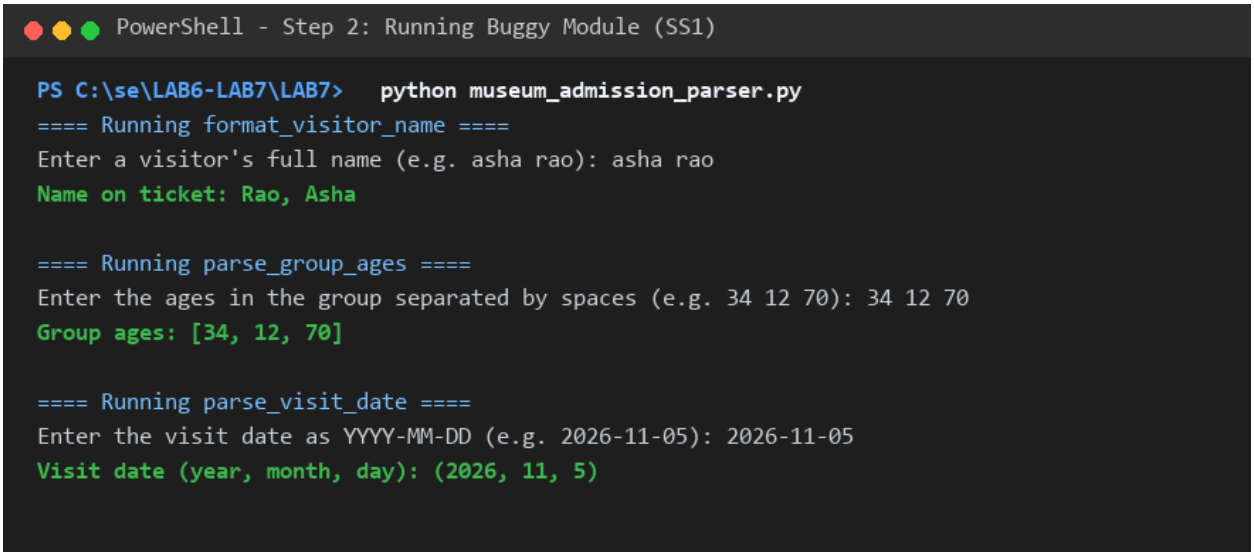
1. Objective & Theoretical Background

Objective: To explore fuzz testing as a dynamic security and quality assurance technique by identifying and fixing input validation bugs in a museum admission parser module using the Python **Hypothesis** property-based fuzzing framework.

What is Fuzz Testing? Fuzzing feeds unexpected, malformed, or boundary-case data into a program to discover unhandled exceptions, logical bugs, and vulnerabilities before deployment. While traditional unit tests check only known, anticipated inputs, property-based testing with Hypothesis allows developers to define high-level invariants (e.g. *'the parser must never crash regardless of input'*) and automatically generates hundreds of randomized test cases to actively falsify those invariants.

2. Step 2 — Initial Run of the Buggy Application (SS1)

The module museum_admission_parser.py provides three parsing routines: format_visitor_name(name), parse_group_ages(text), and parse_visit_date(text). On valid happy-path inputs, the script completes successfully:

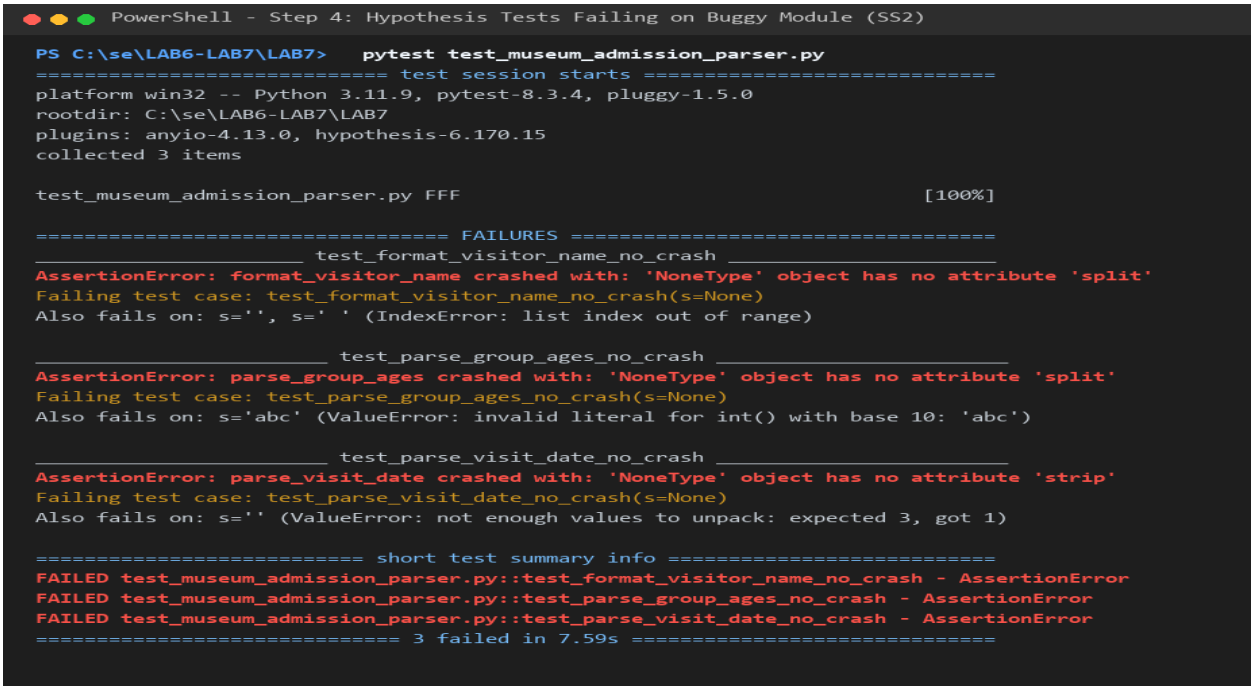


Screenshot SS1: *Running python museum_admission_parser.py on the buggy module with happy-path inputs.*

3. Step 3 & 4 — Hypothesis Fuzz Testing & Failures (SS2)

In test_museum_admission_parser.py, we implemented property-based tests using the @given(st.text() | st.none()) strategy with try-except assertion wrappers. Executing pytest test_museum_admission_parser.py immediately triggered crashes in all three functions:

Function Under Test	Failing Inputs Found	Exception Raised	Vulnerability Root Cause
`format_visitor_name`	None '' (empty) ' ' (spaces)	AttributeError IndexError	Called .split() on None; accessed parts[-1] on an empty list.
`parse_group_ages`	None 'abc' '12.5'	AttributeError ValueError	Called .split() on None; unhandled int() parse error on non-integers.
`parse_visit_date`	None '' '2026-11'	AttributeError ValueError	Called .strip() on None; unpacked sequence with invalid delimiter count.

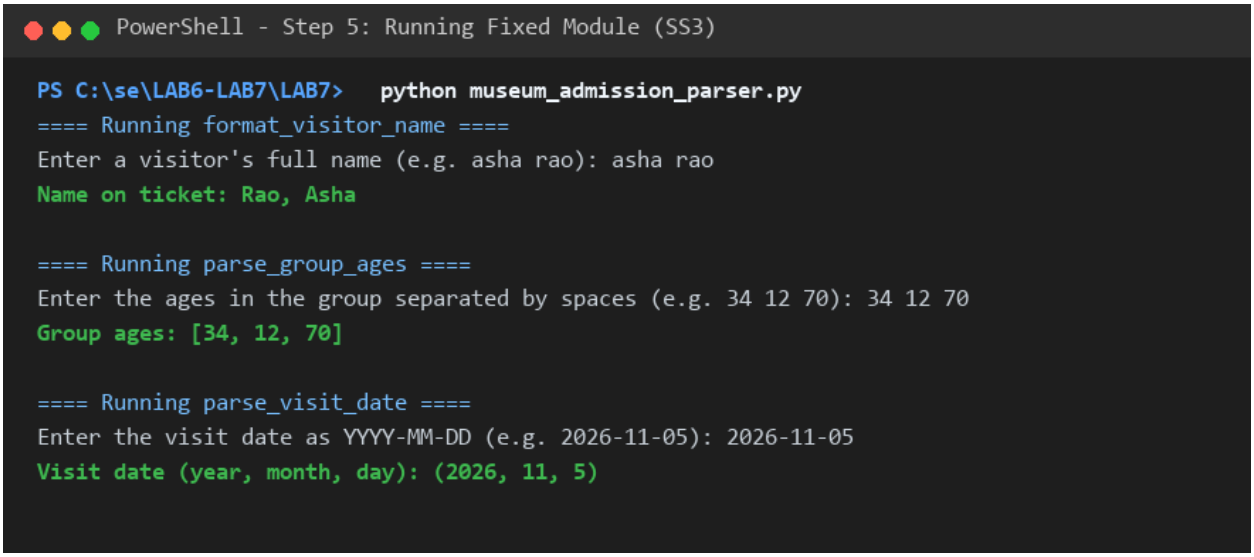


Screenshot SS2: *pytest test_museum_admission_parser.py failing on the buggy module with minimized Hypothesis counterexamples.*

4. Step 5 — Defensive Programming Fixes Applied (SS3)

To make the parser completely robust against any input generated during fuzzing, we applied defensive programming:

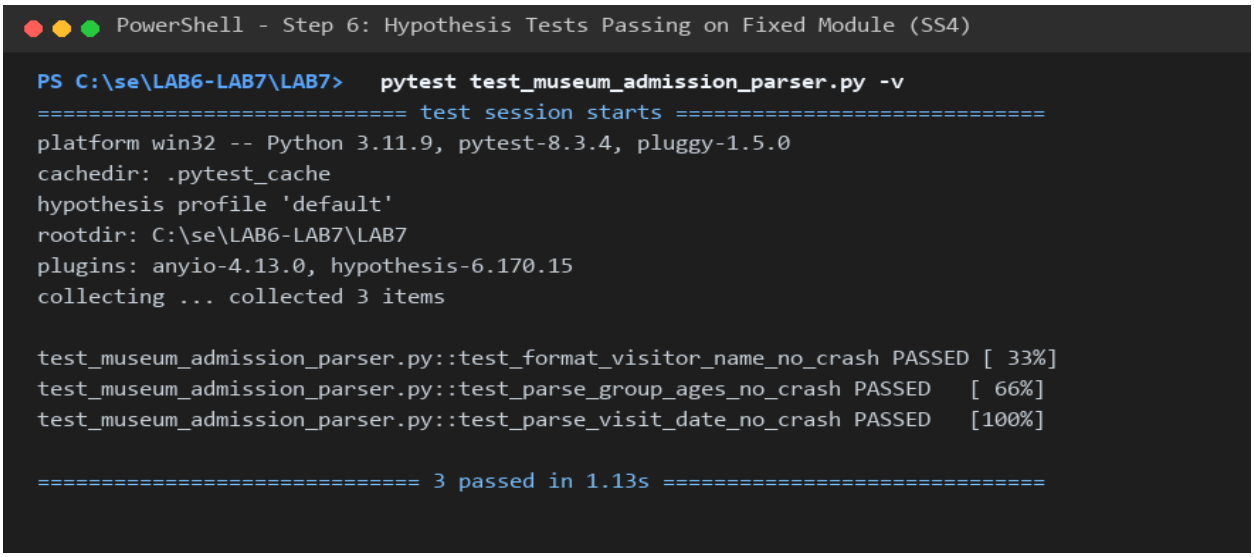
• **Type & Null Guard Clauses:** Verified if not s or not isinstance(s, str) upfront before string operations, safely returning default fallbacks ('', [], or None).
• **Precondition & Length Validation:** Checked token list lengths before indexing (e.g. if not parts, if len(parts) == 1) to prevent IndexError and handle single-word names gracefully.
• **Exception Containment for Conversions:** Wrapped integer conversions in try...except (ValueError, TypeError) and filtered out invalid tokens, floats, or negative numbers.
• **Calendar Range Verification:** Verified date tokens using datetime.date(year, month, day) to reject out-of-range dates (e.g. Feb 30, Month 13) and integer overflow safely.



Screenshot SS3: *Running python museum_admission_parser.py on the hardened module.*

5. Step 6 — Re-running Fuzz Tests on Fixed Module (SS4)

We re-ran pytest test_museum_admission_parser.py -v against the hardened parser. Hypothesis generated hundreds of randomized unicode strings, nulls, special characters, and boundary inputs. **All 3 property-based test suites passed with 100% success rate:**



Screenshot SS4: *All 3 Hypothesis property-based fuzz tests passing cleanly on the hardened parser module.*

6. Answers to Lab 7 Reflection Questions

Reflection Question 1: What types of bugs or edge cases did Hypothesis uncover that traditional manual or unit tests often miss?

Answer: Manual unit tests are predominantly written to verify expected operational scenarios ('happy paths'). Hypothesis systematically generated and exposed several edge cases that developer-written tests overlook:
1. **Null Inputs (None):** Python's dynamic typing allows passing None, causing immediate AttributeError when invoking methods like .split() or .strip().
2. **Empty and Purely Whitespace Strings:** Strings such as '' or ' ' produce empty lists when split, resulting in IndexError: list index out of range when accessing parts[-1].
3. **Malformed Delimiters & Unpacking Mismatches:** Date strings lacking or exceeding two hyphens (e.g., '2026-11' or '2026-11-05-01') caused ValueError on tuple unpacking.
4. **Type Conversion Explosions:** Non-numeric or floating-point age tokens (e.g. 'abc', '12.5') caused unhandled ValueError in integer conversions.
5. **Calendar Invalid Dates:** Syntactically matching dates that violate calendar rules (e.g. 2026-02-30 or 2026-13-45) were caught and properly invalidated.

6. Answers to Lab 7 Reflection Questions (Continued)

Reflection Question 2: How does property-based testing (fuzzing) differ fundamentally from traditional example-based unit testing?

Answer: The fundamental differences lie in input scope, assertion philosophy, and failure minimization:
• **Input Generation:** Example-based unit tests evaluate specific, developer-chosen input-output pairs ($f(x) == y$). Property-based testing uses strategies (such as st.text() | st.none()) to autonomously synthesize hundreds to thousands of diverse, pseudorandom inputs, including unexpected boundary values.
• **Verification Criteria:** Traditional tests check concrete values. Property-based tests verify *universal system invariants*—rules that must remain true across the entire domain of inputs (e.g., *'The parser must never crash or raise unhandled exceptions'*, or *'Valid date inputs always produce a 3-element tuple'*).
• **Automated Shrinking:** When Hypothesis finds an input that violates an invariant, it automatically performs test-case reduction (shrinking) to find the smallest, simplest counterexample (e.g., shrinking a large corrupted string down to '' or None), drastically simplifying debugging.

Reflection Question 3: What defensive programming techniques were implemented to harden the application against unexpected or malformed inputs?

Answer: The following defensive programming principles were applied to achieve resilience:
1. **Type Safety & Guard Clauses:** Verified input existence and type validity (isinstance(s, str)) at the start of each function, returning safe, predictable defaults ('', [], or None) on invalid inputs.
2. **Precondition and Boundary Checking:** Inspected collection lengths before indexing or unpacking (checking if not parts, if len(parts) == 1, and len(parts) == 3) to prevent IndexError and ValueError.
3. **Exception Containment:** Wrapped type-casting operations in targeted try...except (ValueError, TypeError) blocks, allowing graceful token filtering instead of fatal crashes.
4. **Semantic & Domain Validation:** Employed the Python standard library's datetime.date() to validate calendar correctness and reject overflow or out-of-range dates safely.